

Parte 2. Comprensión del flujo colaborativo

Responde esta parte en el documento parte_2/parte_2.pdf. Incluye en cada punto el planteamiento y tu explicación. }

1. Flujo colaborativo

Supón que quieres colaborar con el repositorio de otro desarrollador. Ordena y explica los siguientes elementos:

Push, Fork, Pull Request, Clone, Merge, Commit, Review, Branch y Modificar archivos. Agregando cualquier operación que consideres necesaria.

Respuesta:

Para colaborar de manera ordenada, limpia y segura en un proyecto ajeno utilizando Git y githud, el flujo de trabajo estándar sigue esta secuencia lógica:

Fork: Es una operación que se realiza directamente en la plataforma web de githud. Consiste en crear una copia exacta del repositorio original de otro desarrollador dentro de tu propia cuenta personal de githud. Esto te otorga un espacio propio en la nube donde tienes permisos totales de escritura.

Clone: Es un comando de Git (`git clone <url>`) que descarga una copia local del repositorio que tienes en tu cuenta (el fork) hacia el almacenamiento físico de tu computadora, permitiéndote trabajar de manera local en tu entorno de desarrollo (como Visual Studio Code).

Branch (Creación de rama): Consiste en crear una nueva rama de trabajo (`git checkout -b <nombre-rama>`) separada de la rama principal (main). Esto es fundamental para aislar los cambios o la nueva característica que vas a desarrollar sin alterar o corromper el código base principal.

Modificar archivos: Es la acción de escribir, editar, corregir o eliminar código y documentación dentro de tu entorno local para cumplir con el propósito de tu colaboración.

Commit: Es el proceso mediante el cual guardas formalmente los cambios realizados en el historial local de Git (`git commit -m "mensaje"`). Cada commit representa un punto de control lógico que describe claramente qué modificaciones se hicieron y por qué.

Push: Es la instrucción (`git push origin <nombre-rama>`) que envía los commits almacenados en tu computadora hacia tu repositorio remoto en githud (tu fork), sincronizando tu rama de trabajo en la nube.

Pull Request (PR): Es una solicitud formal que abres desde tu interfaz de githud hacia el repositorio original del propietario. Su objetivo es proponer que los cambios desarrollados en tu rama sean evaluados e integrados al proyecto principal.

Review (Revisión): Es la etapa donde el propietario del repositorio original (o los mantenedores designados) inspecciona el código propuesto en el Pull Request, analiza los

archivos modificados, ejecuta pruebas y puede dejar comentarios, sugerencias o solicitar cambios (Request Changes).

Merge (Fusión): Es la acción final en la que, una vez aprobado el Pull Request, el propietario integra formalmente los commits de tu rama dentro de la rama principal del repositorio original, uniendo oficialmente tu trabajo al proyecto.

Operación adicional: Sync Fork (Sincronización): Consiste en actualizar periódicamente tu repositorio bifurcado (Fork) con los cambios recientes que el repositorio original haya recibido mientras tú desarrollabas tu tarea, evitando conflictos futuros.

2. Fork y Clone

Analiza la afirmación: "Clone crea una copia del proyecto dentro de mi cuenta de githud". Indica si es correcta y explica la diferencia entre Fork y Clone.

Respuesta:

La afirmación es incorrecta.

Clone (git clone): Es un comando de Git que se ejecuta en tu terminal local para descargar una copia de un repositorio remoto (ya sea tuyo o de un tercero) directamente en el almacenamiento físico de tu computadora, permitiéndote trabajar sin conexión y gestionar los archivos de forma local. No genera ningún registro ni repositorio dentro de tu cuenta de githud.

Fork: Es una acción nativa de la plataforma githud que replica un repositorio público alojado en la cuenta de un usuario para alojarlo como un repositorio independiente dentro de tu propia cuenta de githud.

En resumen, Fork opera en la nube (en los servidores de githud) creando un repositorio remoto en tu perfil, mientras que Clone opera en tu máquina local descargando los archivos directamente a tu disco duro.

3. Pull Request

Supón que realizaste Fork, Clone, Branch, Modificar, Commit y Push. Responde: ¿los cambios ya forman parte del repositorio original? ¿Qué debe ocurrir para incorporarlos?

Respuesta:

No, los cambios todavía no forman parte del repositorio original.

Hasta ese punto, tus modificaciones únicamente residen en tu repositorio remoto personal (el Fork que creaste en tu cuenta de githud y en la rama que enviaste). Para que el propietario del repositorio original considere y finalmente incorpore tus aportaciones, deben suceder los siguientes pasos:

1. Debes abrir un Pull Request (PR) desde tu repositorio hacia el repositorio original utilizando la interfaz web de github.
2. El propietario o mantenedor del proyecto debe realizar una revisión detallada de los cambios propuestos.
3. El propietario debe aprobar y autorizar la operación de Merge (fusión) para que los archivos pasen a formar parte oficial del código fuente del repositorio original.

4. Request Changes

El propietario revisa tu Pull Request y selecciona Request Changes. Explica qué debes hacer, si necesitas crear otro Pull Request y qué ocurre cuando realizas nuevamente push.

Respuesta:

- Qué debes hacer: Debes leer detenidamente los comentarios, observaciones y correcciones que el propietario dejó en el Pull Request. Posteriormente, en tu entorno de desarrollo local (posicionado en la misma rama asociada al PR), debes realizar las modificaciones de código solicitadas.
- ¿Necesitas crear otro Pull Request?: No. No es necesario ni correcto crear un nuevo Pull Request. El Pull Request actual permanece abierto y está vinculado dinámicamente a la rama que estás utilizando.
- Qué ocurre cuando realizas nuevamente push: Una vez corregido el código en tu máquina local, realizas un nuevo commit con los cambios y ejecutas git push hacia tu repositorio remoto. github detectará automáticamente la actualización de esa rama, añadirá el nuevo commit al historial del Pull Request existente y notificará al propietario para que continúe con la revisión.

5. Merge y repositorio local

Un Pull Request fue aceptado y se realizó Merge en github. Sin embargo, el repositorio local del propietario no contiene los cambios. Explica por qué sucede y qué operación debe realizarse.

respuesta:

Esto sucede porque el repositorio local del propietario y el repositorio remoto en github son entornos separados. Aunque el Merge se ejecutó de forma exitosa en la nube (servidores de github), la computadora del propietario no se actualiza de manera automática. Su copia local sigue manteniendo exactamente el estado que tenía antes de la fusión.

Qué operación debe realizarse: El propietario debe posicionarse en su rama principal local (main) y ejecutar el comando de actualización:

```
Bash
git pull origin main
```

Esto descargará e integrará los últimos cambios desde el repositorio remoto de github hacia su equipo local, sincronizando ambos entornos.

6. Sync Fork

Tu Fork fue creado varios días atrás y el repositorio original recibió nuevos commits. Explica qué herramienta utilizarías, qué repositorio se actualiza y qué diferencia existe entre Sync Fork y git pull.

Respuesta:

- Utilizará la opción "Sync Fork" (Sincronizar bifurcación) integrada directamente en la interfaz gráfica de la plataforma web de github.
- Qué repositorio se actualiza: Se actualiza tu repositorio remoto personal (el Fork que tienes alojado en tu cuenta de github), incorporando los commits más recientes que el autor original aplicó en su proyecto base.
- Diferencia entre Sync Fork y git pull:
 - Sync Fork es una acción que se ejecuta en la nube (en la interfaz de github) para mantener sincronizado tu repositorio remoto bifurcado respecto al repositorio original del cual se originó.
 - Mientras que git pull es un comando de la terminal que descarga e integra los cambios desde un repositorio remoto (ya sea github) hacia tu entorno de desarrollo local (en tu computadora).